

TRI 论文研究进展与投稿可行性报告

拟投稿: AAAI 2027 Main Technical Track

英文题目: Updating the World Is Not Rebinding the Target: Temporal Referent Integrity in Tool-Using Agents

中文题目: 更新世界状态不等于重新绑定操作目标: 工具型智能体中的时序指称完整性

2026 年 7 月 21 日

摘要

本报告系统梳理时序指称完整性 (Temporal Referent Integrity, TRI) 研究的问题定义、方法设计、实验闭环、现有证据、适用边界与投稿风险。研究关注一个容易被普通任务成功率掩盖的错误: 工具型智能体已经根据用户指令选定了具体操作对象, 但在刷新数据库、邮箱或任务列表后, 又把原来的选择条件应用到新状态, 进而把操作悄悄转移到另一个对象。本文认为, 环境事实发生变化与用户授权改变操作目标是两个不同的决策。

当前研究已经完成受控主实验、独立状态复制、三模型强基线、真实数据库写入、人工语义验证、公开基准审计和组合压力测试。结果表明, 这一错误在受控条件下可以跨模型复现, 并会造成对错误数据库实体的真实修改; 在刷新前明确记录“保持已经绑定的身份”还是“刷新后重新执行查询”, 可以显著减少该类错误。现有证据支持将论文定位为工具型智能体中“绑定之后的指称转换授权”机制诊断与评测盲区研究, 但尚不足以推断真实部署中的发生频率, 也不能支持通用安全保证。

关键词: 工具型智能体; 时序指称完整性; 实体绑定; 状态更新; 目标转换授权; 诊断评测

一、研究概况与当前结论

论文的核心研究闭环已经跑通, 方法主线已经确定, 主要实验、强基线、消融、真实写入后果、人工验证和外部边界检查均已有了结果。研究不只证明“模型答错了”, 而是依次回答了四个更关键的问题: 错误是否发生在正确初始选择之后; 错误是否来自目标转换授权而不是记忆丢失; 简单方法的收益究竟来自哪里; 错误是否会落实为真实工具操作。基于这些问题, 当前已经形成下面这条完整证据链:

发现问题 -> 严格定义 -> 构造最小对 -> 复现现象 -> 提出方法 -> 对比和消融 -> 验证真实错误后果 -> 人工语义验证 -> 第三模型复制 -> 强基线检验 -> 外部 benchmark 审计 -> 明确适用边界。

换言之, 这项工作已经从现象观察推进到可复现、可归因、可干预和可审计的阶段。当前最稳妥的论文定位是:

一篇关于工具型 Agent 中“正确初始绑定之后, 是否有权改变操作目标”的机制发现、诊断方法和 benchmark 盲区论文。

它不适合包装成“通用 Agent 安全控制器”, 也不能宣称这种错误在真实流量中普遍或高频发生。按照目前证据, 论文已经值得投稿, 但外部有效性和相对 Binding Drift 的概念新颖性仍是决定审稿结果的主要风险。

二、论文到底研究什么

考虑两个指令:

1. “现在找出优先级最高的未读邮件。刷新邮箱, 然后回复它。”
2. “先刷新邮箱。然后找出优先级最高的未读邮件并回复。”

假设刷新前邮件 A 优先级最高, 刷新后邮件 B 变成最高。两个任务经历相同的环境变化, 但正确目标不同:

- 第一个指令已经在刷新前选定 A, 刷新并不授权 Agent 改成 B;
- 第二个指令要求刷新后再选择, 因此应该选 B。

现有 Agent 往往把“状态更新”与“目标重选”混在一起。一个典型执行过程是: 模型先查询工具并得到邮件 A 的稳定 ID; 随后工具返回刷新后的列表, 其中邮件 B 成为新的最高优先级对象; 最后模型根据完整历史生成回复动作。即使历史中仍保留 A 的 ID、实体快照和原选择条件, 模型也可能把“优先级最高”重新当成一条待执行查询, 于是把目标从 A 换成 B。这个错误不是模型忘记了 A, 而是模型没有把 A 表示成一个已经建立、不能被状态刷新自动改写的身份承诺。反过来, 永远锁定 A 又会破坏第二种确实要求刷新后重选的任务。

论文把这个问题定义为 时序指称完整性 (Temporal Referent Integrity, TRI):

在初始目标已经正确绑定后，环境状态更新不能自动改变用户授权的操作对象；Agent 必须判断原描述已经成为固定身份，还是仍然是等待未来执行的查询。

这个问题区别于：

- 首次实体选择错误；
- 普通文本共指；
- 记忆丢失或上下文截断；
- 工具调用顺序错误；
- 目标已经失效时的动作策略。

为了便于理解，报告后文使用以下几个术语：

- 初始绑定：智能体第一次根据用户描述找到具体对象，并获得该对象的稳定 ID；
- 保持型任务 (Preserve)：对象在刷新前已经确定，刷新后仍应操作同一 ID；
- 重选型任务 (Reevaluate)：用户有意把选择推迟到刷新后，应在新状态中重新执行选择条件；
- 条件 TRI 错误：初始绑定正确、旧对象仍存在且可执行、刷新后出现新的同角色赢家，但智能体在没有授权的情况下更换目标；
- 错误实体写入 (wrong-entity write)：模型不仅选错标签，而且真正修改了数据库中不应被修改的那条记录。

三、论文主线是否完整

1. 问题定义已经完成

论文区分了两个决策：

- 世界状态是否更新：刷新后事实当然可以改变；
- 操作目标是否转换：只有用户语义授权时才能改变。

实验中的条件 TRI 只统计以下机会：初始绑定正确、刷新后出现同角色竞争实体、旧实体仍存在且动作有效、Agent 最后却改投新赢家。这四个条件分别排除了首次选错、无可执行旧目标、没有真正竞争对象以及普通工具失败。只有满足这些条件，最终换目标才能被归因为“正确绑定之后的未授权转换”。

因此“受控”主要是为了因果隔离，不是因为实验数量不足。如果只看最终任务是否成功，就无法区分模型究竟是第一次就选错了对象、刷新顺序做错了、旧对象已经失效，还是在正确绑定后擅自换了目标。更多实验可以提高模型和语言覆盖，但不能取消这一严格定义。

2. 主方法已经确定

当前主方法是刷新前目标承诺编译 (Compile-then-act, CTA)。这里的“编译”不是生成程序代码，而是要求控制器在环境变化之前，把自然语言中的时间关系转换成一个明确、可执行的目标状态：

- 如果用户在刷新前已经选定对象，保存具体实体 ID，刷新后继续作用于这个 ID；
- 如果用户要求刷新后再决定，保存选择条件，刷新后重新执行；
- 若已绑定目标失效，则按照明确策略拒绝或重新选择，而不是静默换目标。

例如，“找出当前最早失败的部署，刷新后重启它”会被编译成“保持部署 ID d17”；“刷新后找出最早失败的部署并重启”则会被编译成“刷新后执行选择条件”。后续 actor 不再需要从完整长对话中重新猜测时间语义。我们还实现了类型化生命周期记录和确定性执行保护，但实验表明主要收益来自“刷新前把保持还是重选编译清楚”，执行 gate 只提供额外保护。因此论文没有把复杂控制器包装成主要创新。

3. 方法升级已经完成 Go/No-Go

我们额外实现并测试了事件图和可执行选择器：

- 事件图：Qwen 9/20，GLM 20/20；
- 可执行选择器：Qwen 15/20，GLM 18/20；
- 原目标承诺编译：Qwen 13/20，GLM 20/20。

新方法在两个模型上的方向冲突，也未达到预先冻结的 schema/selector 95% 门槛，因此没有升级为主方法。这个负结果说明方法选择已经形成闭环：我们不是看到新结构复杂就采用，而是按冻结标准淘汰了不稳定升级，保留最简单、证据最稳定的方法。

四、实验闭环完成情况

需要回答的问题	对应实验	当前状态
普通结构化状态是否仍会换错目标	160 任务主实验	已完成，两个模型复现
是不是永远锁定旧目标就能解决	Always-Lock / Always-Reevaluate	已完成，两者均为 60%，错误互补
收益是否只来自多一个字段或提前规划	mode 字段、普通提前计划、validity gate 消融	已完成，均不能稳定达到主方法效果
主要收益来自编译还是执行 gate	目标承诺编译、自由 actor、带 gate 生命周期	已完成，主要收益来自刷新前编译
结果是否依赖原始任务模板	50 条独立人工英文改写	已完成，同方向复现
结果是否依赖最初状态和 schema	240 任务、40 个新状态簇、10 个新 schema	已完成，独立复制
是否只在两个模型上成立	DeepSeek 第三模型全量复制	已完成
是否是 Generic ledger 人为诱发	三模型普通完整历史和强语义提醒	已完成
错误是否会真正修改错误对象	模型面对 SQLite + 预测写入重放	已完成
人类是否认同保持/重选语义	3 人盲标 100 条	已完成
公开 benchmark 是否已经覆盖	ToolSandbox、App-World、tau3 审计	已完成
单目标方法能否直接组合到多指称	multi-refresh / multi-referent 压力实验	已完成，发现明确边界
官方邻近方法能否公平比较	Binding Drift 离线源码审计与语义映射	离线审计完成；learned self/cross 公平适配待完成

从论文论证角度，最主要的对比、消融、鲁棒性和边界实验已经齐全。剩余实验更多是稳定性或规模补充，不再决定核心结论能否成立。

五、现有实验结果

1. 160 任务主实验：现象和方法均成立

方法	Qwen3.5	GLM-5.1
普通结构化记录	64.4%	71.9%
刷新前目标承诺编译	95.0%	96.2%
类型化生命周期 + gate	98.1%	100.0%

普通结构化记录已经保存了原指令、初始 ID、实体快照、选择条件和动作前置条件，所以它不是故意删信息的弱基线。它代表一种常见的 Agent 设计：把所有相关事实写进状态，再让模型根据最新状态自由决策。结果显示，仅靠这种记录方式，两个模型仍有约三成任务失败；刷新前目标承诺编译把准确率提高到 95% 以上。这回答了论文的第一个问题：信息仍然存在，不代表“这些信息对目标身份意味着什么”已经成为可执行约束。

2. 对称控制：不能永远锁定，也不能永远重选

Always-Lock 表示无论用户怎么说都坚持使用旧 ID，Always-Reevaluate 表示每次刷新后都重新执行选择条件。两者都只有 60%，而且失败任务正好互补：前者能做好保持型任务，却会破坏重选型任务；后者正好相反。这排除了“加一个 entity lock 就够了”的简单解释。正确方法必须理解用户何时已经承诺某个身份，何时有意把选择推迟到刷新后。

3. 消融：主要收益来自刷新前目标承诺编译

- 普通记录只增加“保持/重选”字段：Qwen/GLM 均为 75%；

- 普通非类型化提前规划: 81.2%/70.6%;
 - 单独增加动作有效性 gate: 65.0%/73.1%;
 - 目标承诺编译: 95.0%/96.2%;
 - 生命周期自由 actor: 96.9%/98.1%;
 - 再增加确定性 gate: 98.1%/100.0%。
- 这些消融逐步拆开了方法可能有效的原因。只增加一个“保持或重选”标签并不够，普通提前规划在 GLM 上甚至下降，单独检查动作有效性也无法阻止模型把一个仍然有效的新对象当成目标。因此不能把结果解释为“多写一个 mode 字段”“多调用一次模型”或“只要检查动作有效性”。gate 的增益只有 1.2–1.9 个点，核心是刷新前把身份承诺编译出来。

4. 240 任务独立复制：三个模型均出现相同机制

在 40 个新状态簇和 10 个新 schema 上，只看“初始绑定正确、旧目标仍存在且可执行、刷新后新赢家出现”的严格机会：

模型	普通记录的条件漂移	目标承诺编译
Qwen3.5	43/72	0
GLM-5.1	38/80	0
DeepSeek	59/79	0

三个模型合计 140 次普通记录核心漂移，全部可在 SQLite 中重放为对错误实体的实际写入。这里的分母之所以因模型而不同，是因为条件指标只统计各模型已经正确完成初始绑定的任务，而不是事后删除不利结果；完整端到端准确率和初始绑定率在论文中同时报告。这个结果说明 v3 主实验不是某一组模板、某几个状态或两个模型的偶然现象。

为进一步检查 240 条任务是否只是由某个领域或模板家族主导，我们补充了逐组剔除分析：每次完整删除一个领域，或完整删除一个模板家族，然后重新计算 CTA 相对普通记录的配对准确率差值。Qwen 在逐领域剔除后的差值范围为 18.5–25.5 个点，逐模板剔除为 20.4–26.3；GLM 分别为 23.1–25.9 和 20.8–26.1；DeepSeek 分别为 15.7–22.2 和 14.2–19.6。所有剔除结果都保持为正，说明主结果不是由单一领域或单一措辞模板造成的。该分析仍不能把模板任务等同于自然分布数据，但降低了伪重复和局部模板主导的风险。

5. 三模型强基线：完整历史和事后提醒仍不够

- 我们补充了两个更强、也更接近普通 Agent 的基线：
- 普通完整历史：两轮调用，保留完整对话，不提示 TRI；
 - 最终语义提醒：最终一步明确要求模型判断保持、重选或拒绝。

模型	普通完整历史	最终语义提醒	目标承诺编译
Qwen3.5	63.3%	69.6%	70.8%
GLM-5.1	67.1%	80.8%	94.2%
DeepSeek	68.8%	75.8%	91.2%

最终提醒确实有帮助，说明模型能够从明确提示中部分恢复正确语义。但在每个模型的 80 个“刷新后赢家改变”的保持型任务中，仍分别发生 56、38、42 次目标替换。这意味着把提醒放在刷新之后仍然太晚：模型已经同时看到旧选择条件和新的赢家，需要重新解释二者的关系。目标承诺编译相对提醒在 GLM 和 DeepSeek 上显著更高；Qwen 上只高 1.2 个点，置信区间跨 0，因此我们不会宣称 CTA 在所有模型总准确率上都显著优于强提示。

更关键的机制区别是：完整历史和最终提醒没有一个独立、可审计的刷新前绑定事件，所以只能统计无条件目标替换；目标承诺编译明确暴露初始绑定，并在初始绑定正确后的三个模型中都实现 0 条件漂移。

6. 错误后果：不是标签答错，而是真实错误写入

新补充的六组完整历史实验共 1,440 个任务，全部完成且没有最终 API 或解析失败。把输出重放到 SQLite 后：

模型	普通完整历史 writes	wrong	最终语义提醒 writes	wrong
Qwen3.5	87		70	
GLM-5.1	79		46	
DeepSeek	75		57	

六组共出现 414 次错误实体写入。例如，模型本应更新先前选定的工单 A，却把更新写入刷新后排名更高的工单 B；数据库最终状态因此发生了可观察的错误，而不是只在答案文本中出现一个错误 ID。此外，早期 40 条模型直接操作 SQLite 的实验中，普通记录在 Qwen/GLM 上分别产生 13/8 次错误写入，生命周期方法均为 0。这组实验把论文结论从“指称判断错误”连接到了“真实工具动作后果”。

7. 人工语义和自然改写

- 3 名不参与研究设计的标注者盲标 100 条原始或改写指令；
- Fleiss kappa = 0.708；
- 多数票与 benchmark gold 一致率为 86%；
- 50 条独立人工英文改写上，普通记录为 60%/74%，目标承诺编译为 90%/98%。

这说明“刷新前已经选定对象”和“刷新后再选择”的区别具有人类可操作性，也不只存在于作者模板中。需要诚实保留的边界是：目标失效后的 Reject 策略人类一致性较低，因此论文核心应放在 Preserve/Reevaluate，而不是把 Reject 当作自然语言共识。

8. 多指称压力实验：方法边界已经暴露

在包含两次刷新、监控对象、操作对象和无关工具调用的任务中，标量生命周期方法没有稳定胜过 Generic。加入 action target 与 monitoring reference 的角色索引后，Qwen 从 35/40 提高到 39/40，但 GLM 与简单方法持平。

因此当前方法只应主张适用于“控制器编排的、单刷新、单操作目标”场景；多指称组合仍是后续问题。这个负结果已经写入论文，不会隐藏。

六、外部验证告诉了我们什么

我们审计了未修改的公开任务和轨迹：

- ToolSandbox: 129 个语义任务家族，严格原生 TRI 机会为 0，1 个 near-match；
- AppWorld: 732 个任务、244 个生成家族，1 个自然 near-match；公开轨迹没有观察到正确绑定后的目标替换；
- tau3-bench: 2,449 个任务、10,832 条公开轨迹，严格原生机会为 0；
- 低干预 AppWorld 工具闭环：28 次正确且及时的绑定中，条件 TRI 为 0；两个错误发生在首次选择之前，属于工具顺序错误。

这些结果不是“现实中 TRI 很常见”的证据，也不能简单解释为“TRI 不存在”。更准确的解释是，公开任务很少给模型制造一个可以观察 TRI 的机会：任务通常没有在正确绑定之后插入独立状态变化，或者没有让同角色的新对象成为竞争者，或者最终评分只检查任务完成而不检查具体修改了哪个实体。因此论文的外部贡献应表述为：发现现有评测对绑定后目标转换授权的覆盖盲区，而不是估计真实发生率。

审计采用六项 checklist：先前同角色绑定、独立状态变化、刷新后竞争赢家、稳定 ID、后续可执行写入、可评分错误后果。结果分为 strict opportunity、near-match 和 excluded 三层；ToolSandbox 为 0/129 strict、1/129 near-match。需要说明的是，这一公开数据审计是单人元数据复核，不是第二标注者一致性实验；所有场景、字段和排除理由已保存在 artifact 中。审稿时应把它称为可复核的 coverage audit，而不是无争议的 prevalence ground truth。

为了进一步确认现象不只存在于作者自建的 JSON 标签环境，本轮还复核了一个冻结的、与 ToolSandbox 兼容的 24 任务试验。该实验保留 ToolSandbox 原生的 reminder 数据库以及查询、修改工具，只在任务语义和中途同步事件上加入 TRI 所需的状态变化。严格条件下，GLM 的普通结构化控制器在 6 个可判定机会中有 3 次未授权换目标，而匹配的 Stable 控制为 0/2；Qwen 的自由生命周期 actor 为 2/6，确定性 gate 重放后降为 0/6。

这项结果构成从受控诊断到真实数据库和工具接口的“桥梁证据”：当外部工具环境中确实出现 TRI 机会时，错误可以发生。但它仍不是官方 ToolSandbox 排行榜结果，也不能说明该错误在自然任务中的频率。原因是任务和同步事件经过自定义干预，严格审计是在已有输出后进行，而且只覆盖 6 个选择器状态簇。因此论文会把它称为“与公开基准兼容的干预证据”，而不是确认性外部效应量。

七、与最邻近工作的关系

最接近的工作是 Binding Drift。我们已经审计其官方代码和离线 benchmark:

- Entity Lock 与本文 Always-Lock 基本对应，只能处理保持型任务；
- 官方确定性 re-verification 路径读取 gold target，只能作为 oracle upper bound；
- TRI 同时包含“必须保持”和“必须重选”的对称任务，研究的是用户是否授权指称转换，而不只是实体是否被替换。

两项工作都关心实体在多次执行中的稳定性，因此存在明显重叠。关键区别在于：Binding Drift 主要询问“先前绑定的实体是否在后续步骤中漂移，以及如何重新验证原请求”；TRI 进一步询问“世界变化后，用户是否授权把一个已经解析完成的身份承诺重新变回查询”。在 TRI 中，保持旧目标和刷新后重选都可能是正确答案，仅靠锁定或重新验证原查询无法同时解决这两个方向。投稿时不能回避这种关系，应该把差异集中在绑定后的转换授权、对称动态控制和严格条件分母上。

可以把两项工作放在一个二维问题地图中理解。第一个维度是初始绑定是否正确，第二个维度是环境变化后是否应该重新求值。当初始绑定错误时，如何重新验证并阻止错误传播是 Binding Drift 更擅长的问题；当初始绑定已经正确时，究竟应该保持旧身份还是执行新的选择查询，才是 TRI 的核心问题。本文并不声称 CTA 是更强的通用 Binding Drift 防御，也不要求它在初始绑定错误的工作流上胜过 re-verification。论文需要证明的是：现有锁定和重新验证思路不能自动回答“用户是否授权转换”这一独立决策。

本轮评审建议要求进一步实践 Binding Drift 的 practical reverify。官方完整源码副本已确认位于 `external_sources/bin` 当前 commit 为 `0e040e0954b18d4621a6f9b16f6e6e9591c822e1`；该目录被 `.gitignore` 排除，不进入 TRI 提交包。官方 runner 绑定 AWS Bedrock, self/cross reverify 的核心逻辑是使用原始 step-1 请求和当前候选实体再调用一次模型。硅基流动不能直接运行该 runner，需要保持 prompt、状态和评分逻辑不变，只替换 Bedrock 调用层。该结果必须标注为 author adaptation，不能称为官方 LLM sweep；oracle reverify 仍不得进入 learned 主表。

该 author adaptation 的 20-task 对称 smoke 已完成。任务覆盖 10 个领域，每个领域包含一组共享相同状态、selector 和动作的 Preserve/Reevaluate Flip 配对。Entity Lock 类比方法在 Preserve 上 10/10、Reevaluate 上 0/10；GLM self-reverify 正好相反，为 0/10 和 10/10，说明无条件锁定与无条件重新解析分别只解决授权问题的一半。Qwen reverify 只有 3/20，其中 14 条选择了状态中存在但不满足 selector 的其他实体，因此主要是 selector grounding 错误，不能解释成纯粹的时间授权倾向。冻结 CTA 在 Qwen/GLM 上分别为 12/20 和 17/20，是两个模型中唯一同时在 Preserve 和 Reevaluate 上获得非零正确率的方法。

这组结果是当前最直接的最近邻比较，但仍有严格边界：官方 verifier 通常接收精简 step1_referent，而 TRI 必须保留完整时序指令，否则会删除“是否授权重选”这个研究变量。因此它是保留官方 prompt 框架的接口适配，不是官方 Binding Drift 分数，也不测试初始绑定错误修复。两个 verifier run 均为 20/20 完整、0 API/解析错误、0 retry，共使用 16,088 tokens。

八、论文当前完成度

模块	完成度	说明
问题定义和形式化	90%	TRI、条件分母、授权不变量已完成
主方法	95%	CTA 已锁定；复杂升级已按门槛淘汰
主实验	100%	v3 两模型完整
独立复制	100%	v7 三模型完整
对比与消融	95%	强提示、完整历史、对称控制和表示消融齐全
错误后果	100%	模型直接写入和 1,440 次重放均完成
人工验证	100%	3 人 100 条盲标、50 条独立改写完成
外部边界	90%	三个公开 benchmark 审计和 AppWorld case study 完成
论文正文	85%	已有 7 页正文 + 1 页参考文献的可编译版本，仍需 reviewer pass
补充材料和证据溯源	85%	主要报告、原始输出和 provenance 已建立

这里的完成度不是接受概率。论文闭环已完成，但最终质量仍取决于叙事聚焦、相关工作定位和是否严格

限制主张。

九、当前优势与硬风险

优势

1. 问题直观，最小对可以快速说明核心矛盾；
2. 有严格的正确初始绑定条件，能把机制与其他错误分开；
3. v3 发现和 v7 独立复制形成两阶段证据；
4. 三个模型、人工改写和新 schema 均支持相同方向；
5. 有真实数据库 wrong write，不只是分类准确率；
6. 对称控制和完整消融能够定位主要收益来源；
7. 强完整历史和语义提醒基线已经补齐；
8. 外部零结果和多指称负结果均保留，论文主张较克制。

硬风险

1. 外部有效性：自然 benchmark 没有观察到条件 TRI，无法证明真实部署中高频发生；
2. 方法新颖性：目标承诺编译本身简单，论文价值必须来自问题发现、因果诊断和 benchmark blind spot；
3. 相关工作重叠：需要让审稿人相信 TRI 相对 Binding Drift 的授权视角和动态对称设计具有实质差异；
4. 合成任务占比高：人类改写、held-out schema 和 SQLite 缓解但不能消除该问题；
5. 组合性有限：多刷新、多角色场景尚未得到跨模型稳定解决；
6. Qwen 强基线持平：CTA 不能宣称在所有模型总体准确率显著胜过后置语义提醒。